Nesne Yönelimli Programlama – Konu Anlatımı

Modüller, Cohesion ve Coupling

Slaytlar, ders kitabı (Schach 8e), ders videoları ve WhatsApp grubu temel alınmıştır

9 Kasım 2026

4 Modüllerden Nesnelere: Cohesion ve Coupling (Chapter 7)

! Hoca derste vurguladı Hafta 1, 4 ve 5

"The most important chapter is chapter seven. It describes how modules should be decomposed." Eski vize notu: *"Cohesion (7) ve Coupling (5)'i açıklayınız. Türlerinin başlıklarını yazınız."* 2025 WhatsApp: *"Derste kodu verip hangi türde cohesion ya da coupling olduğunu sorarım demişti"*, *"vizede veriyor ama başka bir seviyeye dönüştürme isteyebilir"*. Son iki vizede bu sorudan **kod verilip tür bulma ve daha iyi hâle getirme** istendi.

4.1 Neden modülerleştirme?

Hocanın sorusu: "Kitaplarını rafta neden kategorilere ayırırsın?" Yazılımı alt modüllere bölmek **yönetimi** ve **bakımı** kolaylaştırır (*divide and manage*): hata kolay bulunur, regression etkisi olmadan düzeltilir. Ama **her modülerleştirme iyi tasarım değildir.** İyi bir yazılım için iki şey gerekir: (1) iyi bir yaşam döngüsü modeli (**iterative and incremental**) ve (2) **robust architecture**: **high cohesion, low coupling** ve SOLID ilkeleri.

4.2 Modül nedir?

Module: *A lexically contiguous sequence of program statements, bounded by boundary elements, with an aggregate identifier.*

- **Lexically contiguous:** kodda bitişik (ardışık) satırlar.
- **Boundary elements:** sınır öğeleri; C tabanlı dillerde `{ ... }`, Pascal'da `begin ... end`.
- **Aggregate identifier:** modülün tamamına verilen **isim**.

Hocanın derste sorduğu soru

`if (kosul) { s1; s2; } else { s3; s4; }` bloğu bir modül müdür? **Hayır.** Ardışık ve sınır öğeleriyle çevrili ama **adı (aggregate identifier) yok**; ismiyle çağrılamaz. Bir metodun içine konup metoda isim verilirse modül olur. Metotlar, sınıflar, struct'lar modüldür.

4.2.1 Bilgisayar tasarımı örneği

Beceriksiz bir mimar ALU, shifter ve 16 register'dan oluşan bir bilgisayarı AND, OR, NOT kapılarıyla kuracaktır.

- **Tasarım 1:** işleve göre üç çip: register'lar bir çipte, ALU bir çipte, shifter bir çipte.
- **Tasarım 2:** kapı türüne göre üç çip: bütün AND kapıları bir çipte, OR'lar bir çipte, NOT'lar bir çipte.

İkisi işlevsel olarak eşdeğerdir ama tasarım 2: **anlaşılması zor**, **hatanın yerini bulmak zor** (ALU çıktısı yanlışsa hangi kapı?), **genişletmesi zor** (belleği büyütmek için her çipe kapı ekleyip bağlantıları yeniden kurmak gerekir), **başka üründe yeniden kullanılamaz**. Modüller tasarım 1 gibi olmalıdır:

maximal relationships within modules, minimal relationships between modules

- **Cohesion:** bir modülün **içindeki** etkileşimin derecesi (*degree of interaction within a module*). **Yüksek** olmalı.
- **Coupling: iki modül arasındaki** etkileşimin derecesi (*degree of interaction between two modules*). **Düşük** olmalı.

Hocanın uyarısı

"Our friends are confused about cohesion and coupling. Cohesion is within a module. Coupling is between modules." Kohezyon **tek bir modülün** içini, coupling **iki modülün** ilişkisini anlatır. Sınavda "robust architecture" sorusunun cevabı: **loose (low) coupling / high cohesion**.

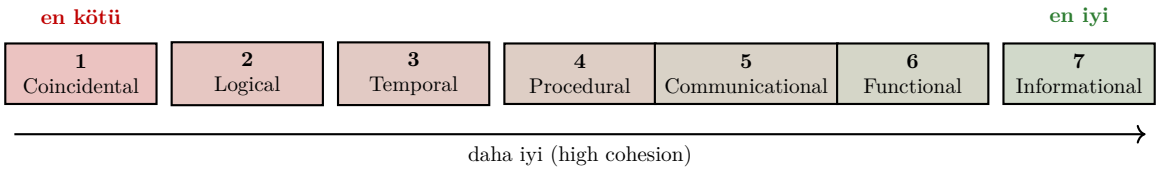
4.2.2 Bir modülün function, logic ve context'i

Örnek: "Newton algoritmasıyla double precision sayıların karekökünü hesaplayan modül."

- **Function** (ne yapar): karekök hesaplar.
- **Logic** (nasıl yapar): Newton algoritmasıyla.
- **Context** (ne üzerinde çalışır): double precision sayılar.

Klasik (C/SD) yaklaşımda modülün adı işlevidir: `compute_square_root` (alt çizgili Pascal notasyonu); nesne yönelimli dillerde `ComputeSquareRoot` (camel/Pascal case) kullanılır.

4.3 Cohesion (modül içi uyum): 7 seviye



Ölçek doğrusal değildir (non-linear). Kitapta 6 ve 7 aynı seviyede gösterilir; hoca 7'yi (informational) en iyi olarak sayar.

1. Coincidental cohesion (tesadüfi) — en kötü

Tanım: Modül **birbiriyle tamamen ilgisiz** birden çok eylem yapar.

Kitap örneği: `print_next_line, reverse_string_of_characters_comprising_second_parameter, add_7_to_fifth_parameter, convert_fourth_parameter_to_floating_point`.

Nasıl oluşur? "Her modül 35 ile 50 satır arasında olacak" gibi kurallardan: uzun modüller bölünür, kısa parçalar başka modüllere eklenir.

Neden kötü? Bakımı zorlaştırır (tasarım 2 gibi spagetti), **yeniden kullanılamaz**.

Düzeltme (kolay): modülü her biri tek iş yapan ayrı modüllere böl (SOLID'in **S**'si: Single Responsibility).

```
void Islemler(string s, ref int sayac, string x)
{
    Console.WriteLine("Sonraki satir");          // yazdirma
    char[] d = s.ToCharArray(); Array.Reverse(d); // string ters cevirme
    sayac += 7;                                   // sayaca 7 ekleme
    double sonuc = Convert.ToDouble(x);           // ondaliga cevirme
}
```

2. Logical cohesion (mantıksal)

Tanım: Modül **birbiriyle ilgili** bir dizi eylem yapar; bunlardan **biri çağıran modül tarafından seçilir** (opCode / flag / switch).

```
void Rapor(int opCode)          // cagiran, hangi raporun basilacagini secer
{
    if (opCode == 1)      YerelSatisRaporuYazdir();
    else if (opCode == 2) BolgeselSatisRaporuYazdir();
    else if (opCode == 3) UlusalSatisRaporuYazdir();
    else                  UluslararasiSatisRaporuYazdir();
}
```

Kitap örnekleri: `function_code = 7; new_operation(op_code, dummy_1, dummy_2, dummy_3);` (dummy değişkenler kullanılmıyor); bütün giriş/çıkışı yapan bir nesne; OS/VS2'de 13 farklı eylem yapan, arayüzünde 21 parametre bulunan modül.

Neden kötü? (1) **Arayüz anlaşılması zor** (hangi opCode'da hangi parametre kullanılıyor?), (2) birden fazla eylemin kodu **iç içe geçebilir (intertwined)**, (3) yeniden kullanmak zordur. Hoca: yeni bir lazer yazıcı eklemek için `if (opCode == 38)` eklemek gerekir; bu **Open-Closed ilkesini** ihlal eder ("genişletmek için mevcut kodu değiştiriyoruz"). Kitap: yeni bir teyp birimi eklenince lazer yazıcı kodu etkilenebilir.

Düzeltme: her eylemi kendi metoduna ayır (functional); çağıran doğrudan ilgili metodu çağırsın. Nesne yönelimli çözüm: ortak bir `IRapor` arayüzü ve her rapor için ayrı sınıf (polymorphism); yeni rapor eklemek mevcut kodu değiştirmez.

3. Temporal cohesion (zamansal)

Tanım: Modül **zaman açısından ilişkili** bir dizi eylem yapar: sadece *aynı anda / art arda yapılmaları gerektiği için* bir aradadırlar (tipik: başlatma ve kapanış işlemleri).

Kitap örneği (`perform_initialization`): `open_old_master_file, new_master_file, transaction_file, print_file; initialize_sales_district_table; read_first_transaction_record; read_first_old_master_record`.

```
void ProgramiBaslat()
{
    dosya = File.Open("ana.dat", FileMode.Open); // dosya ac
    satisBolgeTablosu.Clear();                   // tabloyu sifirla
    ilkKayit = OkuIlkKayit();                    // ilk kaydi oku
    Console.WriteLine(DateTime.Now);             // zamani yaz
}
```

Neden kötü? Eylemler birbirleriyle **zayıf**, ama **başka modüllerdeki** eylemlerle **güçlü** ilişkilidir (ör. `sales_district_table` başka modüllerde güncellenir; tablo değişirse bu modül de değişmelidir). Yeniden kullanılamaz.

Düzeltme: her başlatma işini ilgili modüle/sınıfa taşı (tablo başlatma tablo sınıfının constructor'ına, dosya açma dosya sınıfına).

4. Procedural cohesion (işlemsel)

Tanım: Modül, **ürünün izlediği prosedürle (iş akışıyla) ilişkili** bir dizi eylem yapar. Kitap örneği: `read_part_number_and_update_repair_record_on_master_file`.

```
void ParcaNoOkuVeTamirKaydiGuncelle()
{
    int parcaNo = int.Parse(Console.ReadLine()); // parca numarasini oku
    TamirKaydi k = anaDosya.Bul(parcaNo);        // farkli veri: tamir kaydi
    k.TamirSayisi++;
    anaDosya.Guncelle(k);
}
```

Temporal'dan iyidir (eylemler ürünün işleyişi açısından ilişkili), ama eylemler hâlâ zayıf bağlıdır, modül birden çok iş yapar, **yeniden kullanılamaz**.

Düzeltme: `ParcaNoOku()` ve `TamirKaydiGuncelle(int parcaNo)` diye iki functional metoda ayır; akışı çağıran yönetsin.

5. Communicational cohesion (iletişimsel)

Tanım: Procedural ile aynı (*ürünün prosedürüne göre ilişkili eylemler*), **ek olarak bütün eylemler aynı veri üzerinde çalışır**. Kitap örnekleri: `update_record_in_database_and_write_it_to_audit_trail`, `calculate_new_coordinates_and_send_them_to_terminal`.

```
void KaydiGuncelleVeDenetimeYaz(Kayit k)   // iki is, ayni veri: k
{
    veritabani.Guncelle(k);
    denetimIzi.Yaz(k);
}
```

Neden kötü? Hâlâ birden fazla iş yapar; yeniden kullanılabilirlik düşüktür. Hoca'nın Lego benzetmesi: 5 birimlik Lego'yu sadece 5 birime ihtiyaç olduğunda kullanabilirsin; 1 birimlik Lego her yerde kullanılır. Modül tek iş yaptıkça yeniden kullanılabilirliği artar.

Düzeltme: `KaydiGuncelle(Kayit k)` ve `DenetimeYaz(Kayit k)` olarak ikiye böl.

6. Functional cohesion (işlevsel)

Tanım: Modül **tam olarak tek bir eylem** yapar. Kitap örnekleri: `get_temperature_of_furnace`, `compute_orbital_of_electron`, `write_to_diskette`, `calculate_sales_commission`.

```
double SatisKomisyonuHesapla(double satisTutari)
{
    return satisTutari * 0.05;
}
```

Avantajları: daha **yeniden kullanılabilir**; corrective bakım kolay (**hata izolasyonu**, daha az regression fault); ürünü **genişletmek kolay**.

7. Informational cohesion (bilgisel) — en iyi

Tanım: Modül bir dizi eylem yapar; **her eylemin kendi giriş noktası (entry point)** ve **bağımsız kodu** vardır ve **hepsi aynı veri yapısı üzerinde** çalışır. Bu, özünde bir **abstract data type**'tır (ADT); **sınıflar** informational cohesion örneğidir.

```
class IsKuyrugu                          // veri + o veri uzerindeki islemler
{
    private Queue<Is> kuyruk = new Queue<Is>();      // veri gizli
    public void Ekle(Is i)  { kuyruk.Enqueue(i); }   // giris noktasi 1
    public Is Cikar()       { return kuyruk.Dequeue(); } // giris noktasi 2
    public int Uzunluk()    { return kuyruk.Count; } // giris noktasi 3
}
```

Kitap örneği (Figure 7.6): `sales_region_table` verisi ile onun üzerinde çalışan `initialize_sales_region_table`, `update_sales_region_table`, `print_sales_region_table` işlemleri; her birinin kendi giriş ve çıkışı var.

4.3.1 Cohesion türünü bulma algoritması (sınavda koddan tür bulurken)

1. Modül **tek bir eylem** mi yapıyor? ⇒ **Functional**.
2. Birden çok eylem var ve her birinin **ayrı giriş noktası** var, hepsi **aynı (gizli) veri yapısı** üzerinde mi (sınıf/ADT)? ⇒ **Informational**.
3. Eylemler birbirinden **tamamen ilgisiz** mi? ⇒ **Coincidental**.
4. İlgili eylemlerden **biri, dışarıdan gelen bir kod/bayrakla (opCode, if/switch) seçilip** mi çalışıyor? ⇒ **Logical**.
5. Eylemler sadece **aynı zamanda yapıldıkları için** mi (başlangıç/bitiş)? ⇒ **Temporal**.
6. Eylemler ürünün **iş akışında art arda** gelen adımlar mı?
   - Hepsi **aynı veri** üzerinde ⇒ **Communicational**.
   - Farklı veriler üzerinde ⇒ **Procedural**.

Cevapta mutlaka gerekçe yaz

Sadece adı yazmayın: *"It has logical cohesion because the module performs a series of related actions and one of them is selected by the calling module through opCode."* Ardından düzeltilmiş kodu verin ve yeni seviyeyi söyleyin: *"After refactoring each method has functional cohesion."* İki seviye arasında kararsız kalırsanız (ör. coincidental mı temporal mı) hoca da derste *"coincidental ya da temporal"* diye iki cevabı kabul etti; gerekçenizi yazın.

4.3.2 Hocanın derste çözdüğü örnekler

Sıcaklık örnekleri (Hafta 5):

Modül	Cohesion	Gerekçe
compute average daily temperatures at various sites	Functional	tek iş: ortalama hesaplama
initialize sums and open files	Coincidental	ilgisiz işler (hoca; zamana bağlı görülürse temporal)
create new temperature record	Functional	tek iş
read in site, time, and temperature	Procedural	kitapta functional; hoca: "iki ayrı değişken okunuyor, procedural"
edit site, time, or temperature field	Logical	birden çok işlemden biri seçiliyor ("or")
store temperature record	Functional	tek iş
store record for specific site	Functional	tek iş
close files and print average temperatures	Coincidental / Temporal	ilgisiz ama bitişte art arda yapılıyor

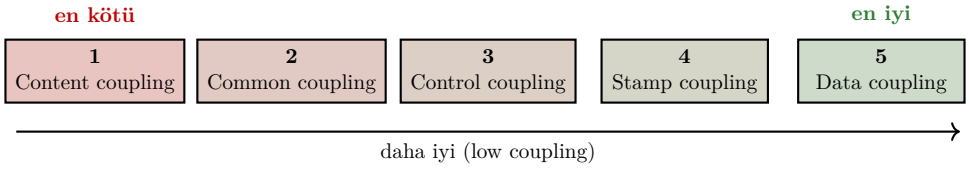
2025 dersinde tahtaya yazılan kod:

```
if (isDoc) UsePrinter("network");
else       UsePdfWriter(new int[] {1, 2, 3, 4, 5});

void UsePrinter(string b)  { Console.WriteLine(b); }                    // functional cohesion
void UsePdfWriter(int[] a) { Console.WriteLine(a[0] + a[a.Length-1]); } // functional
    cohesion
void WriteTimePrint()      { Console.WriteLine(DateTime.Now); Print(true); } // coincidental
    ya da temporal
```

Ek gözlem (coupling): `UsePdfWriter`'a bütün dizi gönderilip yalnızca ilk ve son eleman kullanılıyor ⇒ çağıran ile `UsePdfWriter` arasında **stamp coupling**. `Print(true)` bir bayrak (control element) gönderiyorsa **control coupling**.

4.4 Coupling (modüller arası bağlılık): 5 seviye



1. Content coupling (içerik) — en kötü

Tanım: Bir modül **diğerinin içeriğine doğrudan başvurur**. Örnekler: `p`, `q`'nun bir **komutunu değiştirir**; `p`, `q`'nun **yerel verisine** sayısal bir kayma (displacement) ile başvurur; `p`, `q`'nun içindeki bir **yerel etikete dallanır** (`goto`).

```
modul M2:  ...  BB:  x = x + 1;  ...   // M2'nin icindeki etiket
modul M1:  ...  goto BB;               // M1, M2'nin icine dallaniyor
```

Neden kötü? `q`'daki neredeyse her değişiklik (hatta `q`'yu yeni bir derleyiciyle derlemek) `p`'nin de değişmesini gerektirir; bir modüldeki hata doğrudan diğerini etkiler.

2. Common coupling (ortak)

Tanım: İki modül **aynı global veriye yazma erişimine** sahiptir. Örnekler: `cca` ve `ccb` aynı `global_variable`'ı okuyup değiştirir; iki modül aynı veritabanı kaydını okuyup yazabilir; FORTRAN/COBOL `common`.

```
static class Ortak { public static int durum = 0; }      // global veri

void Modul3() { if (Ortak.durum == 0) Islem1(); else Islem2(); }
void Modul4() { Ortak.durum = 1; }   // Modul3'un davranisini degistirir
```

Neden kötü? Yapısal programlama ruhuna aykırı, kod okunamaz (döngüyü ne bitiriyor?); **yan etkiler**: bir modülü anlamak için global değişkeni değiştiren bütün modülleri okumak gerekir; global değişkenin bildirimi değişince diğer modüller de değişmeli; yeniden kullanmak zor; **clandestine (gizli) common coupling**: `p`'yi hiç değiştirmeden `p` ile ürünün geri kalanı arasındaki bağ değişebilir (Linux çekirdeği örneği); modül gereğinden fazla veriye maruz kalır ⇒ **bilgisayar suçu** riski.

Düzeltme: global değişkeni kaldır; veriyi parametreyle gönder (data coupling) ya da veriyi ve onu değiştiren işlemleri bir sınıfa kapsülle (information hiding).

3. Control coupling (denetim)

Tanım: Bir modül diğerine bir **kontrol öğesi** (control element) gönderir: çağrılan modülün *ne yapacağını* çağıran belirler. Örnekler: **logical cohesion'a sahip bir modüle opCode göndermek**; argüman olarak bir **control switch** göndermek.

```
void M2() { M1(3); }                    // M2, M1'in hangi isi yapacagini secer
void M1(int a)                          // M1 icinde logical cohesion
{
    if (a == 1) Gorev1(); else if (a == 2) Gorev2(); else Gorev3();
}
```

Kitaptaki ayrım: `q`, `p`'ye "*başaramadım*" derse bu **veridir**; "*başaramadım, ABC123 hata mesajını yaz*" derse bu **kontroldür**.

Neden kötü? Modüller bağımsız değildir: çağıran, çağrılanın iç yapısını ve mantığını bilmek zorundadır; yeniden kullanımı azaltır. Genellikle **logical cohesion** ile birlikte görülür.

Düzeltme: her görevi ayrı metot yap; çağıran doğrudan `Gorev3()`'ü çağırsın (her metot functional, aradaki bağ data/yok).

4. Stamp coupling (damga / veri yapısı)

Tanım: Parametre olarak bir **veri yapısı** gönderilir ama çağrılan modül bu yapının **bazı (hepsi değil)** bileşenleri üzerinde çalışır. Örnek: `calculate_withholding(employee_record)`.

```
struct Ogrenci { public int Id; public string Ad, Soyad, KullaniciAdi, Sifre; }

void AdSoyadYaz(Ogrenci o)                 // tum kayit gonderiliyor
{
    Console.WriteLine(o.Ad + " " + o.Soyad); // ama sadece Ad ve Soyad kullaniliyor
}
```

Hocanın vurgusu: **sadece struct değil**: bir **dizi** gönderilip yalnızca bazı elemanları kullanılıyorsa da stamp coupling vardır.

Neden kötü? Kayıttaki hangi alanların kullanıldığı ya da değiştirildiği, modülün tamamı okunmadan anlaşılmaz; yeniden kullanılamaz; **gereğinden fazla veri** geçer: kontrolsüz veri erişimi **bilgisayar suçuna** yol açabilir (hoca: "kullanmadığı hâlde şifreyi gönderiyorsunuz, o modüle şifre yetkisi veriyorsunuz").

Düzeltme: sadece gereken alanları gönder: `AdSoyadYaz(string ad, string soyad)` ⇒ data coupling.

Yapı göndermek her zaman kötü değildir

Yapının **bütün** bileşenleri kullanılıyor veya değiştiriliyorsa sorun yoktur, bu **data coupling**'dir: `invert_matrix(original_matrix, inverted_matrix)`, `print_inventory_record(warehouse_record)`.

5. Data coupling (veri) — en iyi

Tanım: Bütün parametreler **homojen veri öğeleridir**: basit parametreler ya da **bütün elemanları kullanılan** veri yapıları. Örnekler: `display_time_of_arrival(flight_number)`, `compute_product(first_number, second_number)`, `get_job_with_highest_priority(job_queue)`.

```
double AlanHesapla(double en, double boy) { return en * boy; }
```

İçerik, ortak, kontrol ve damga bağlılığının zorlukları yoktur; **bakım kolaydır**.

4.4.1 Coupling türünü bulma algoritması

1. Bir modül diğerinin **kodunu/etiketini/yerel verisini** değiştiriyor ya da ona dallanıyor mu? ⇒ **Content**.
2. İki modül aynı **global** veriyi (static alan, global değişken, aynı DB kaydı) okuyup **yazıyor** mu? ⇒ **Common**.
3. Gönderilen parametre çağrılanın **hangi işi yapacağını belirleyen** bir kod/bayrak mı? ⇒ **Control**.
4. Bir struct/kayıt/dizi gönderiliyor ama **bir kısmı** mı kullanılıyor? ⇒ **Stamp**.
5. Gönderilen her şey basit veri ya da tamamen kullanılan yapı mı? ⇒ **Data**.

4.5 Sınav tipi örnek: kodu verip tür ve düzeltme isteme

2025–26 vizesine benzer örnek

Aşağıdaki metodun cohesion seviyesini bulun, gerekçesini yazın ve daha iyi bir cohesion/coupling ile yeniden yazın.

```
class Siparis
{
    public double toplamTutar;
    public void kdvekle_indirimuygula(int indirimOrani)
    {
        toplamTutar = toplamTutar - toplamTutar * indirimOrani / 100.0; // indirim
        toplamTutar = toplamTutar + toplamTutar * 0.20;                 // KDV
    }
}
```

Çözüm

Cohesion: Communicational. Metot iki ayrı eylem yapıyor (indirim uygulama ve KDV ekleme). Bu eylemler ürünün prosedürüne göre art arda gelen adımlardır (önce indirim, sonra KDV) **ve ikisi de aynı veri** (`toplamTutar`) üzerinde çalışıyor. Tek iş yapmadığı için functional değildir; seçim yapılmadığı için logical değildir.
English: The method has communicational cohesion because it performs two actions (applying the discount and adding VAT) that are related by the procedure of the product and both operate on the same data (toplamTutar).
Düzeltme: Her işi ayrı bir metoda böl (her biri **functional**); veriyi gizle ve işlemleri veriyle birlikte sınıfa koy (sınıf **informational cohesion**); parametreler basit veri (**data coupling**).

```
class Siparis
{
    private double toplamTutar;                        // information hiding
    public Siparis(double tutar) { toplamTutar = tutar; }

    public void IndirimUygula(int indirimOrani)        // functional
    { toplamTutar -= toplamTutar * indirimOrani / 100.0; }

    public void KdvEkle(double kdvOrani)               // functional
    { toplamTutar += toplamTutar * kdvOrani; }

    public double Tutar() { return toplamTutar; }
}
// kullanim:  s.IndirimUygula(10);  s.KdvEkle(0.20);
```

Not: Öğrencilerin bir kısmı "procedural", "logical" veya "temporal" yazdı. Kitaptaki tanıma göre ölçüt **aynı veri üzerinde çalışma** olduğundan doğru seviye *communicational*'dır; "sequential cohesion" Schach'ın 7 seviyesinde yoktur.

4.6 Data encapsulation, ADT ve information hiding

4.6.1 İş kuyruğu örneği

Büyük bir ana bilgisayar için işletim sistemi: yüksek, orta ve düşük öncelikli işler için **üç kuyruk**; iş gönderilince uygun kuyruğa eklenir, çalışmaya hazır olunca kuyruktan çıkarılıp bellek ayrılır.

- **Tasarım 1:** kuyruk işlemleri ürünün her yerine dağılmış ⇒ **düşük cohesion**.

- **Tasarım 2:** `m_encapsulation` modülü: `job_queue` veri yapısı ile `initialize_job_queue`, `add_job_to_queue`, `delete_job_from_queue` bir arada ⇒ **informational cohesion**. Bu, **data encapsulation**'dır: bir veri yapısı ve onun üzerinde yapılan işlemler.

Data encapsulation'ın avantajları:

- **Geliştirme: abstraction**'ın bir örneğidir. Önce ürünü üst düzey kavramlarla (iş kuyrukları ve işlemleri) tasarla, kuyruğun nasıl gerçekleştirildiğini önemseme; sonra alt düzey bileşenleri tasarla, onların nasıl kullanılacağını önemseme. Hoca: *abstraction = gereksiz ayrıntıyı gizleyip ilgili ayrıntıyı vurgulayarak stepwise refinement yapmak.*
- **Bakım:** değişmesi muhtemel yerleri belirle ve değişikliğin etkisini en aza indir. Veri yapıları pek değişmez, gerçekleştirme ayrıntıları değişebilir. Kuyruk iki yönlü bağlı listeye çevrilse bile onu kullanan modüller değişmez, sadece yeniden derlenir.

4.6.2 Abstract data type (ADT)

Tek bir kuyruk yetmez, üç kuyruk gerekir: **veri tipi + o tipin örnekleri üzerinde yapılan işlemler = abstract data type**. Hoca: *ADT, informational cohesion'a sahip veri tipidir; sınıflar ADT örneğidir.*

- **Data abstraction:** tasarımcı ADT düzeyinde düşünür.
- **Procedural abstraction:** bir prosedür tanımlamak = dili genişletmek.

4.6.3 Information hiding

Data abstraction ve procedural abstraction daha genel bir kavramın örnekleridir: **information hiding**. Modüller, **değişmesi muhtemel öğeler gizlenecek** şekilde tasarlanır; gelecekteki değişiklik **yerelleşir** ve diğer modülleri etkileyemez. C#/C++/Java'da `private` alanlar ve `public` metotlar/property'ler ile sağlanır (*en az yetki prensibi*). Public alanlar information hiding'i bozar.

4.6.4 Nesneler, kalıtım, polymorphism ve dynamic binding

- **Birinci inceltme:** ürün ADT'ler cinsinden tasarlanır; değişkenler ("nesneler") ADT'lerin örnekleridir.
- **İkinci inceltme: class** = kalıtımı destekleyen ADT; nesneler sınıfların örnekleridir.
- **Inheritance:** `HumanBeingClass` (age, height, gender); `ParentClass` onun alt sınıfı olup bütün özelliklerini miras alır ve kendininkileri ekler (nameOfOldestChild, numberOfChildren). Kalıtım nesne yönelimli dillerin (Smalltalk, C++, Ada 95, Java, C#) temel özelliğidir; klasik dillerde (C, COBOL, FORTRAN) yoktur. UML'de büyük **boş üçgen** ile gösterilir; aggregation **boş elmas**, association **düz çizgi**.
- **Polymorphism ve dynamic binding:** Klasik paradigmada dosya açmak için doğru fonksiyonu açıkça seçeriz (`open_disk_file`, `open_tape_file`...). Nesne yönelimli paradigmada `myFile.open()` yazılır; hangi `open` metodunun çalışacağı **çalışma zamanında** belirlenir (**dynamic binding**); `open` farklı sınıfların nesnelerine uygulanabilir (**polymorphic**). `checkOrder(b : Base)`, Base'in her alt sınıfının nesnesine uygulanabilir.
- Polymorphism ve dynamic binding hem güçlü hem zayıf yandır: belirli bir metodun birden çok olası sürümü varsa kodu anlamak zorlaşır, bakım olumsuz etkilenebilir.

4.6.5 Nesne yönelimli paradigmanın güçlü ve zayıf yanları

Başarısının nedenleri: veri ve işlemlere genel olarak eşit önem verir; iyi tasarlanmış bir nesne (high cohesion, low coupling) bir fiziksel varlığın bütün yönlerini modeller; gerçekleştirme ayrıntıları gizlidir.

Zayıf yanları: geliştirme eforu ve boyut büyük olabilir (ilk nesne yönelimli proje beklenenden büyük, özellikle GUI varsa; ama sınıflar sonraki projede yeniden kullanılabilir); kalıtım sorun çıkarabilir: **fragile base class problem** (taban sınıftaki değişiklik bütün alt sınıflara dalga gibi yayılır; bu yüzden sınıflar baştan dikkatle tasarlanmalı); alt sınıf ebeveynin bütün özelliklerini miras alır, ağacın altındaki nesneler şişebilir: *"Use inheritance where appropriate"*, gereksiz miras alınan özellikleri hariç tut; polymorphism ve dynamic binding sorun çıkarabilir; kötü nesne yönelimli kod yazmak özellikle kolaydır. Bir gün yerini daha iyisi alacaktır (ör. aspect-oriented programming).

4.7 SOLID ilkeleri (giriş)

Hoca cohesion/coupling'i anlatırken SOLID'i bağladı; ayrıntıları uygulama haftalarında (final konusu) işlenir, ama vizede kod düzeltirken gerekçe olarak yazmak işe yarar:

İlke	Anlamı	Cohesion/coupling ile bağı
Single Responsibility	Bir sınıfın/modülün değişmek için tek bir nedeni olmalı; tek iş.	Coincidental, logical, procedural ve communicational modülleri böl ⇒ functional.
Open-Closed	Genişletmeye açık, değişikliğe kapalı.	Logical cohesion'da yeni durum için `if` eklemek OCP ihlali; arayüz + polymorphism ile çöz.
Liskov Substitution	Alt sınıf, taban sınıfın yerine geçebilmeli, davranışını bozmamalı.	Kalıtımı "where appropriate" kullan.
Interface Segregation	Sınıflar kullanmadıkları metotları içeren büyük arayüzlere zorlanmamalı.	Stamp coupling'e benzer: gereğinden fazlasına bağımlı olma.
Dependency Inversion	Üst düzey modüller somut sınıflara değil soyutlamalara (interface) bağlı olmalı.	Coupling'i düşürür.

4.8 Kısa tanım soruları

Module A lexically contiguous sequence of program statements, bounded by boundary elements, with an aggregate identifier.

Cohesion The degree of interaction within a module. (Seven levels: coincidental, logical, temporal, procedural, communicational, functional, informational.)

Coupling The degree of interaction between two modules. (Five levels: content, common, control, stamp, data.)

Good design High cohesion and low coupling (maximal interaction within a module, minimal interaction between modules).

Data encapsulation A data structure together with the operations performed on that data structure.

Abstract data type A data type together with the operations performed on instantiations of that data type (a module with informational cohesion; a class).

Information hiding Designing modules so that items likely to change are hidden; future change is localized.

Class An abstract data type that supports inheritance.

Dynamic binding The method to be invoked is determined at run time according to the class of the object.

Polymorphism The same method can be applied to objects of different classes, each behaving in its own way.

Fragile base class problem A change to a base class ripples through all of its subclasses.